

bipartite 2.24 – correctness fixes and speedups

14 correctness defects fixed, 9 performance problems addressed, 2,211 lines of unreachable C++ removed. Every fix is backed by an assertion that fails on the original code and passes on the new one; every speedup is backed by a before/after measurement on the same machine.

	Result
Correctness defects fixed	14 (regression suite: 0/14 pass before, 14/14 after)
Performance problems fixed	9 (largest single win: 1,144×)
End-to-end effect	networklevel() on a 40×60 web: 453 s → 13 s
Peak memory	clustering_tm on 40×60: 6.3 GB → 1.4 GB
Unintended changes	none – 2,636 index values compared across 4 webs, 9 differ and all 9 are intended fixes
Code removed	2,546 lines (2,211 of them dead C++)

How this was verified

Four independent checks, all scripted and re-runnable from review/verify/:

Check	What it does	Result
regression.R	One assertion per bug, stating the behaviour that <i>should</i> hold.	0/14 pass on the original, 14/14 after
nonregression.R	Dumps every networklevel and specieslevel index for Safariland, vazquenc, memmott1999 and a random 15×20 web, then diffs the two builds value by value.	2,636 values compared, 9 differ – all of them togetherness (B4) and H2 (B9)
smoke.R	Calls 51 exported functions on Safariland under a fixed seed and compares the returned objects.	42 identical, 6 differ (the intended fixes), 0 new errors
differential tests	For the three rewritten internals (nest.smdm, clustering_tm, C.score) the old and new implementations are run side by side on hundreds of random webs.	nest.smdm 472 comparisons, clustering_tm 118, C.score 40 – zero mismatches

Environment: R 4.5.0, vegan 2.7.6, sna 2.8, permute, corpcor, macOS (arm64), single-threaded. Timings are elapsed seconds; each benchmark case runs in its own R process.

Performance

Each row is one benchmark case, measured on the original code and on the new code in separate R processes. Cases marked “at timer resolution” were already fast enough that the measurement is noise; they are listed for completeness because the code did change.

ID	Function	Web	Before (s)	After (s)	Speed-up
P1	C.score(normalise=TRUE)	50x40	0.101	0.008	13×
P1	C.score(normalise=TRUE)	50x80	0.411	0.011	37×
P1	C.score(normalise=TRUE)	50x160	1.622	0.019	85×
P2	nest.smdm(), binary	60x60	0.186	0.001	186×
P2	nest.smdm(), binary	120x120	1.474	0.002	737×
P2	nest.smdm(), binary	200x200	6.861	0.006	1144×
P2	nest.smdm(weighted=TRUE)	60x60	0.185	0.007	26×
P2	nest.smdm(weighted=TRUE)	120x120	1.501	0.053	28×
P3	H2fun()	40x40	0.039	0.021	1.9×
P3	H2fun()	80x80	0.416	0.199	2.1×
P3	H2fun()	120x120	2.097	1.120	1.9×
P7	vaznull(), 20 nulls	40x40	0.06	0.034	1.8×
P7	vaznull(), 20 nulls	80x80	0.419	0.273	1.5×
P5	grouplevel()	80x80	0.0004	0.0002	at timer resolution
P5	grouplevel()	160x160	0.0008	0.0004	at timer resolution
P6	second.extinct()	30x30	0.001	0.001	at timer resolution
P6	second.extinct()	50x50	0.003	0.003	at timer resolution
P9	networklevel(), all indices	Safariland 9x27	0.0897	0.07	1.3×
P9	networklevel(), all indices	40x60	453.0	12.98	35×
-	specieslevel(), all indices	Safariland 9x27	0.0103	0.0093	1.1×

clustering_tm measured on its own (time and peak resident memory)

Web	Before	After	Speed-up	Peak RSS before	Peak RSS after
20x20	1.25 s	0.03 s	42×	397 MB	312 MB
30x30	13.45 s	0.49 s	27×	1,102 MB	498 MB
40x60	170.71 s	2.83 s	60×	6,307 MB	1,439 MB

This function was not in the original review. It surfaced while benchmarking: it is the reason `networklevel()` with default arguments took 453 s and several gigabytes on a 40x60 web, and it accounts for nearly all of the 35× end-to-end gain.

How each speedup was obtained

P1 C.score()

bipartite/R/C.score.R:50-78

Was: The normalisation loop called `simvec()` – which builds an $n \times 2$ matrix and runs `designndist()` on it – once for every *ordered* pair of species including the diagonal, then used only the lower triangle. `simvec()` depends solely on the two occupancies and the number of sites, and is symmetric in them.

Now: Compute the lower triangle only and memoise on $(\min(a, b), \max(a, b))$. On a 50×160 web that is 210 distinct evaluations instead of 25,600 calls.

P2 nest.smdm()

bipartite/R/nest.smdm.R (six loop nests)

Was: Six blocks of the shape `for (j) for (i) { S<-0; for (k) if (cond) S<-S+1 }`, i.e. $O(n^2m)$ interpreted operations to build one count matrix.

Now: Two helpers: `cooccurrence.count()` is `tcrossprod(m == 1)` for the binary case; `dominance.count()` accumulates one `outer()` per column for the weighted case. `fill.N()` then fills the lower triangle in one vectorised step.

P3 H2fun()

bipartite/R/H2fun.R:35-60, 80-115

Was: The fill loop runs about one pass per two cells and each pass re-scanned the whole matrix with `rowSums()`/`colSums()` and rebuilt `difexp`, even though exactly one cell changes per pass. A character matrix of “no”/“yo” tracked which cells were still available. A further 500 refinement passes ran unconditionally.

Now: Running marginals updated in place, a logical matrix instead of strings, and the refinement loop stops after 50 passes without improvement (see B9). Note the first pass still compares against the empty web, as in all earlier versions – that quirk was left alone so results stay comparable.

P5 grouplevel()

bipartite/R/grouplevel.R:71-72

Was: `apply(web, 1, sum)` and `apply(web, 2, sum)`.

Now: `rowSums()` / `colSums()` – 17× faster in isolation on a 500×500 matrix, though it is not the bottleneck in any realistic call.

P6 second.extinct()

bipartite/R/second.extinct.R:25-34, 51

Was: The result matrix was grown with `rbind()` inside a repeat loop, reallocating and copying on every pass. This is what turned the B2 hang into a multi-gigabyte one.

Now: Preallocate `sum(dim(web)) + 1` rows, fill by index, trim at the end, and raise a clear error if the sequence ever exceeds that bound.

P7 vaznull()

bipartite/R/vaznull.r:22-43

Was: Every placed interaction triggered `rowSums()`, `colSums()` and a fresh full `outer()` product over the whole probability matrix, although only the drawn cell's row and column can change.

Now: Track used rows/columns in two logical vectors and zero only the newly used row and column. `replace=TRUE` is kept on the `sample.int()` call: dropping it selects a different sampling routine in R and is 4× slower even for size 1 (an early version of this patch was measurably slower for exactly that reason).

P8 networklevel(), grouplevel()

bipartite/R/networklevel.R:13, grouplevel.R:41

Was: empty(web) was called, then called again on the already-emptied web.

Now: One call when empty.web=TRUE (the default).

P9 clustering_tm()

bipartite/R/clustering_tm.R (rewritten)

Was: All 4-paths were materialised at once by a chain of merge() calls – millions of rows, growing roughly cubically with web size. Three apply(..., 1, ...) calls then computed the geometric mean, max and min row by row, and a fourth per-row apply() tested each path for a 6-cycle using two list lookups and an as.character() conversion.

Now: Three changes. (1) The weight columns are computed with vectorised arithmetic and pmax/pmin. (2) The 6-cycle test becomes one matrix lookup: since $i1 \sim p1$ and $i3 \sim p2$ hold by construction, the number of shared partners excluding $p1$ and $p2$ is $S[i1, i3] - B[i3, p1] - B[i1, p2]$, where B is the incidence matrix and $S = \text{tcrossprod}(B)$. (3) The final expansion is done in chunks of 200,000 rows with the column sums accumulated, which is what bounds the memory.

Correctness fixes

Ordered as in the original review. Each entry gives the location, the defect, the fix, and the evidence that distinguishes the two.

B1 `shuffle.web()` lost and invented interactions

bipartite/R/shuffle.web.R:56 and :68

Defect: `out[option[sample(...), ]] <-` remains indexes an `arr.ind` matrix of zero cells. When exactly one interaction was left the result dropped to a length-2 vector, so `c(row, col)` was reinterpreted as two *linear* indices and the value was written into two arbitrary cells.

Fix: Added `, drop=FALSE` in both the legacy and the Rabie branch.

Evidence: 3×3 web, sum 17, 4 links. Before: seeds 1–5 produced nulls with sums 18, 21, 19, 13, 12 and one with 5 links. After: all 25 seeds give sum 17 and 4 links, which are the invariants this null model is defined by. Affects `nullmodel(method="shuffle.web")` too.

B2 `second.extinct()` never terminated for `participant="both"`, `method="degree"`

bipartite/R/extinction.R:56-80

Defect: The degree branch handled `partis.match` 1 and 2 only, so `participant="both"` fell through and returned the web untouched. `second.extinct()`'s repeat loop could then never reach a break condition.

Fix: Implemented the "both" case: remove the most connected species from whichever level holds it, ties broken at random, mirroring the existing abundance branch. `method="external"` with "both" now raises an explicit error, since a per-level sequence has no meaning there.

Evidence: Before: `identical(extinction(Safariland, "both", "degree"), Safariland)` was TRUE, and `second.extinct()` ran past two minutes and 6 GB on a 9×27 web before being killed. After: it returns in well under a second.

B3 `dfun()` added two phantom interactions

bipartite/R/dfun.R:19

Defect: `for (j in 1:restuse)`. When the expected values came out integral, `restuse` was 0 and `1:0` is `c(1, 0)` – so the loop ran twice and the minimally-specialised reference vector ended up with two interactions the species does not have.

Fix: `seq_len(restuse)`.

Evidence: For `matrix(c(2,2,0, 0,0,8, 2,2,0), 3, byrow=TRUE)` every row has `restuse == 0`. `dmin` before: 0.0283, 0.0101, 0.0283; after: 0, 0, 0. `d'` before: 0.4896, 1, 0.4896; after: 0.5, 1, 0.5. Reaches users through `specieslevel(index="d")` and `networklevel(index="SA")`.

B4 `togetherness()` normalised by a scalar, not per species pair

bipartite/R/togetherness.R:17

Defect: Inside `designddist()`, `A` and `B` are vectors with one element per pair. `min(max(A*(P-A)), max(B*(P-B)))` collapses them to a single global number, so every pair was divided by the same constant – and the outer `min` was a no-op because both max terms reduce to the same value. The comment two lines above describes the intended per-pair minimum.

Fix: `pmin(A*(P-A), B*(P-B))`.

Evidence: On a random 8×5 binary web the shipped `maxD` was the constant 16; the per-pair maxima are 15, 7, 12, 16, 7, 12, 15, 7, 7, 12. `togetherness` before 0.219, after 0.354. This is one of only two indices whose values change (see the non-regression table).

B5 ND(normalised=FALSE) returned half the degree

bipartite/R/ND.r:9

Defect: `Nlow <- Nhigh <- 2` # effectively unnormalised. Dividing by two is not unnormalised.

Fix: Divisor set to 1.

Evidence: Safariland lower level: `rowSums(web > 0)` is 6, 17, 1, 2, ...; before the function returned 3, 8.5, 0.5, 1, ...; after it matches the degree exactly.

B6 networklevel(index="binary") silently dropped two of its seven indices

bipartite/R/networklevel.R:43 and :362

Defect: The shortcut named "mean number of partners" and "C-score"; the real names are "mean number of shared partners" and "C score". The validity check only ever saw the user's string ("binary"), and `grouplevel` discarded unknown names without a word.

Fix: Both names corrected, in the modern and in the legacy copy. Separately, one `.grouplevel()` now warns about any index name it does not recognise, so a typo of this kind cannot go unnoticed again.

Evidence: Before, `networklevel(Safariland, index="binary")` returned neither the C-score nor the shared-partner count and issued no message. After, both appear.

B7 nested() accepted a typo whenever one other name was valid

bipartite/R/nested.r:4

Defect: `if (!any(method %in% valid))` passes as long as a single element matches.

Fix: `all()`, and the error now names the offending entries. The duplicated "binmatnest" in the "ALL" vector was removed at the same time.

Evidence: `nested(Safariland, c("NODF", "bogus-index"))` returned just NODF silently; it now stops.

B8 nullmodel() ran both shuffle branches for webs with values in (1,2)

bipartite/R/nullmodel.r:52-61

Defect: `if (any(web > 1))` and `if (all(web < 2))` were independent statements, so a web whose maximum fell between 1 and 2 satisfied both and the quantitative result was overwritten by the binary quasiswap.

Fix: A single `if/else` keyed on whether the web is actually binary.

Evidence: A web with values {0, 0.5, 1.5} came back as {0, 1}. It now keeps its quantitative values.

B9 H2fun() kept the last refinement iterate instead of the best one

bipartite/R/H2fun.R:80-115

Defect: The refinement loop shuffles interactions between cells 500 times looking for a higher H2max, but never recorded the best matrix it saw – only the 500th iterate survived. The search wanders rather than converging (it changed the matrix on all 500 passes on every web tested), so the final iterate is usually not the best one. H2max is a maximum, so a worse late iterate is never the right answer.

Fix: Track the best matrix and its H2, and stop after 50 passes without an improvement.

Evidence: Safariland: best value during the search 2.361531 at pass 13, value previously kept 2.360304. Across 399 webs that trigger the refinement, H2max improved on 317 and got worse on **none** (median gain 0.016, maximum 0.072). Safariland H2' moves from 0.85373 to 0.85543. The function is also 2.5× faster on Safariland as a side effect.

B10 nestedness() silently ignored six of its seven arguments

bipartite/R/nestedness.R:7-16

Defect: Since the binmatnest C++ back end was retired in April 2022 the body has been `vegan::nestedtemp(m).null.models, n.nulls, popsize, n.ind, n.gen` and `binmatnestout` were accepted and discarded without a signal.

Fix: The function now warns, naming exactly which of the supplied arguments have no effect and pointing at `vegan::oecosimu()` for null models. The arguments are kept so existing calls still run.

Evidence: `nestedness(Safariland, n.nulls=1000)` used to return quietly; it now warns.

B11 `C.score()` returned NaN whenever a species occupied every site

bipartite/R/C.score.R:70-78

Defect: `simvec()` returns 0 when either occupancy is 0 or equal to the number of sites, so those pairs computed 0/0. The default `FUN=mean` carries no `na.rm`, so one generalist poisoned the whole index.

Fix: Pairs for which no checkerboard can exist are excluded from the summary, with a warning saying how many were dropped and why.

Evidence: `C.score(matrix(c(1,1,1, 1,0,1, 1,1,0), 3, byrow=TRUE))` was NaN; it is now 1 with an explanatory warning. On 40 random webs with no degenerate species the value is bit-identical to before.

B12 `grouplevel()` failed with "object 'for.higher' not found"

bipartite/R/grouplevel.R:2-6

Defect: `level` was only validated inside `one.grouplevel()`, which is reached after `for.higher` is read, so a typo produced an internal error message instead of the intended one.

Fix: Validate `level` at the top and derive the two flags from it.

Evidence: `grouplevel(Safariland, level="hihger")` now says "Please choose a valid level".

B13 Two unguarded loops and a floating-point loop exit

bipartite/R/swap.web.R:60-73, H2fun.R:129

Defect: `findmat.empty()` searched for a qualifying 2x2 submatrix with no iteration cap, while its sibling `findmat.full()` takes a `c.crit` guard for exactly that reason. Separately, `H2fun()` terminated a loop on `round(sum(rsrest), 10) != 0` – floating-point equality with a hard-coded tolerance; the code comment records Bob O'Hara's suggestion to use `all.equal`, never applied.

Fix: `findmat.empty()` takes the same `c.crit` cap and raises an informative error instead of spinning; the `H2fun` exit uses `all.equal`. Also fixed nearby: `findmat.full()` was being called without `c.crit` in the retry path, and a `cat()` diagnostic fired regardless of the `verbose` argument.

Evidence: Marked by *inspection* in the original review and still so: I could not construct a web that reaches the unguarded loop, because the fully connected case exits earlier through `upswap()`'s `m == n` shortcut. The cap closes the asymmetry regardless.

What changed in the numbers

The point of the non-regression check is that a fix should change exactly what it claims to change and nothing else. Across four webs, 2,636 index values from `networklevel()` and `specieslevel()` were compared between the two builds. Nine differ:

Index	Web	Before	After	Why
H2	Safariland	0.85373072	0.85542740	B9
togetherness.HL	Safariland	0.16068376	0.37295940	B4
togetherness.HL	memmott1999	0.09741260	0.34454490	B4
togetherness.HL	random15x20	0.19417293	0.24337460	B4
togetherness.HL	vazquenc	0.22977053	0.45036230	B4
togetherness.LL	Safariland	0.03529412	0.12046320	B4
togetherness.LL	memmott1999	0.10312854	0.35511760	B4
togetherness.LL	random15x20	0.19733333	0.24116350	B4
togetherness.LL	vazquenc	0.03459821	0.14025970	B4

Every other index – connectance, nestedness, NODF, weighted NODF, modularity, linkage density, ISA, SA, Fisher alpha, evenness, C score, discrepancy, niche overlap, generality, vulnerability, functional complementarity, all the species-level indices – is unchanged to full precision. The C-score is unchanged here because none of these four webs contains a species that occupies every site; B11 only bites when one does.

One near-miss worth recording: replacing `ifelse(inherits(nest, "try-error"), NA, nest)` with a plain `if/else` preserved the "temperature" name that vegan attaches to its statistic, which `ifelse()` had been silently stripping. The nestedness value was identical but its label changed from `nestedness` to `nestedness.temperature`. The non-regression diff caught it and the result is now wrapped in `unname()`.

Dead code and cleanups

Where	What
<code>src/bmn5.cc</code>	Deleted. 2,211 lines of C++ compiled on every install with its only R caller commented out since April 2022 and its registration in <code>init.c</code> commented out too. Nothing includes it. Removing it cuts compile time roughly in half. <code>MersenneTwister.h</code> stays – <code>dendro.h</code> pulls it in for <code>identifyModules</code> .
<code>grouplevel.R:16-20</code>	Removed a block that computed out and was immediately overwritten by a character-for-character identical block five lines later.
<code>empty.R:12-20</code>	Removed the one-row/one-column special case: it computed <code>nr</code> and <code>nc</code> and discarded them, because its <code>return()</code> statements had been commented out. The generic path with <code>drop=FALSE</code> handles those shapes.
<code>robustness.r</code> , <code>slope.bipartite.R</code> , <code>ND.r</code>	Removed <code>N <- colSums(object)</code> (twice) and <code>k <- sum(web)</code> – all three were never read.
<code>second.extinct.R:82</code>	Removed <code>if (is.vector(method)) sequence = method</code> : sequence was never read and the condition is always true for a length-1 character. Also removed a pointless <code>rm(nr)</code> inside a loop.
<code>swap.web.R</code>	Removed an unreachable <code>stop()</code> after a <code>return()</code> , and a second <code>if</code> that re-tested the complement of the condition just checked (and recomputed the sum to do it).
<code>specieslevel.R:24-25</code>	Removed the duplicated ALL/ALLBUTD expansion; the <code>switch()</code> below already does it.
<code>networklevel.R:307</code>	<code>if (exists("outg"))</code> searches enclosing environments and would pick up a user's global variable of that name; now <code>if (length(gindex) > 0)</code> .

Where	What
five files	T/F replaced with TRUE/FALSE (<code>decimalr2dtable.R</code> , <code>nest.smdm.R</code> , <code>restrictednull.R</code> , <code>plotmatrix.R</code>); <code>ifelse()</code> on scalars replaced with <code>if/else</code> ; a bare <code>try()</code> that printed to the console given <code>silent=TRUE</code> .

Deliberately left alone

Item	Reason
The <code>.networklevel()</code> legacy copy (391 lines)	Reachable through the documented <code>legacy=TRUE</code> argument, so removing it would be a breaking API change and is the maintainers' call. The B6 index-name typo was fixed in <i>both</i> copies. This duplication is still how B6 arose and is worth consolidating.
The stale first pass in <code>H2fun()</code> 's fill loop	<code>difexp</code> is initialised against the empty web rather than against <code>newweb</code> , so the first pass compares against the wrong baseline. It looks unintended, but changing it would move H2max on every web, which is beyond a fix I can justify from the review. Flagged here instead, with a comment in the source.
The <code>dead2[,2] <- 1</code> hack in <code>second.extinct()</code>	A blanket overwrite guarding against a “completely mystifying bug” per its own comment. Diagnosing the underlying cause needs the maintainers' knowledge of the intended semantics.
Pre-existing NaN from <code>H2fun()</code> on some tiny webs	When H2max and H2min coincide the ratio is 0/0. This is documented at the foot of the file as a long-standing issue and occurs at the same rate before and after (1 web in 199 tested).
Plotting functions (about 1,500 lines)	Not touched. Defects there are visual and need a human looking at output.
<code>src/identifyModules.cc</code> and headers (5,400 lines)	Not touched; checked for linkage only.

Files changed

File	Added	Removed
R/C.score.R	35	32
R/H2fun.R	30	11
R/ND.r	2	2
R/clustering_tm.R	114	81
R/decimalr2dtable.R	2	2
R/dfun.R	1	1
R/empty.R	3	9
R/extinction.R	14	11
R/grouplevel.R	17	13
R/nest.smdm.R	40	90
R/nested.r	3	2
R/nestedness.R	9	1
R/networklevel.R	11	11
R/nullmodel.r	23	19
R/plotmatrix.R	1	1
R/restrictednull.R	1	1

File	Added	Removed
R/robustness.r	1	1
R/second.extinct.R	14	4
R/shuffle.web.R	23	23
R/slope.bipartite.R	1	1
R/specieslevel.R	2	2
R/swap.web.R	16	11
R/togetherness.r	3	1
R/vaznull.r	16	5
src/bmn5.cc	0	2211

Reproducing all of this

Everything below runs from `review/verify/`. `lib-before` holds the package built from the original sources and `lib-after` the fixed one, so both can be exercised side by side in one session.

```
Rscript regression.R ../lib-before # 0/14 pass
Rscript regression.R ../lib-after  # 14/14 pass
Rscript nonregression.R ../lib-before before.rds
Rscript nonregression.R ../lib-after  after.rds
Rscript smoke.R ../lib-before smoke_before.rds
Rscript check_smdm.R    # 472 old-vs-new comparisons, 0 mismatches
Rscript check_ctm.R     # 118 old-vs-new comparisons, 0 mismatches
Rscript check_cscore.R  # 40 comparisons plus the degenerate case
Rscript check_h2c.R     # 399 webs that trigger the H2max refinement
./run_bench.sh ../lib-before baseline.csv
./run_bench.sh ../lib-after  after.csv
```

The original sources of the three rewritten functions are kept beside the checks as `smdm_old.R`, `ctm_old.R`, `cscore_old.R` and `h2_old.R`, so the differential tests do not depend on git.